SDEV 2401

Rapid Backend App Development

REST API Development - 8

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- Adding User Ownership to a Model
- Custom ViewSet Actions for Related Data
- Using Depth in Serializers (and Its Pitfalls)
- Custom Nested Serializers for Clean Output
- Filtering Querysets by Authenticated User
- Adding Search Functionality with Filter Backends
- Key Takeaways

WE ARE ESSENTIAL TO ALBERTA
NAIT

Adding User Ownership to a Model

- Assigning a user to a model (e.g., `Recipe`) allows you to track which user created each item.
- This is useful for filtering, permissions, and personalizing data.

```
class Recipe(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        blank=True,
        null=True
    )
    name = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)
```

- This pattern is common for user-generated content and enables per-user data access.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Custom ViewSet Actions for Related Data

- You can add custom endpoints to a ViewSet using the `@action` decorator.
- For example, to get all comments for a specific blog post:

```
from rest_framework.decorators import action

class BlogPostViewSet(viewsets.ModelViewSet):
    @action(detail=True, methods=['get'], url_path='comments')
    def comments(self, request, pk=None):
        post = self.get_object()
        serializer = CommentSerializer(post.comments.all(), many=True)
        return Response(serializer.data)
```

- This creates a `/blogposts/{id}/comments/` endpoint for related data.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Using Depth in Serializers (and Its Pitfalls)

- The `depth` option in DRF serializers can quickly include related objects.
- For example, to include all ingredients in a `Recipe`:

```
class RecipeDetailSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ['id', 'name', 'ingredients']
        depth = 2
```

- Be careful: `depth` can expose sensitive data and may include more than you want. Use it for quick prototyping, not for production APIs.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Custom Nested Serializers for Clean Output

- Instead of using `depth`, create custom serializers for related objects.
- For example, to show only certain fields for each ingredient in a recipe:

```
class IngredientSimpleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Ingredient
        fields = ['id', 'name', 'quantity']

class RecipeDetailSerializer(serializers.ModelSerializer):
    ingredients = IngredientSimpleSerializer(many=True, read_only=True)
    class Meta:
        model = Recipe
        fields = ['id', 'name', 'ingredients']
```

- This gives you full control over the output and avoids leaking sensitive data.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Filtering Querysets by Authenticated User

- Limit data exposure by filtering querysets to the current user.
- For example, only show a user's own recipes:

```
class RecipeViewSet(viewsets.ModelViewSet):
    def get_queryset(self):
        return Recipe.objects.filter(user=self.request.user)
```

- This is essential for privacy and security in multi-user applications.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Adding Search Functionality with Filter Backends

- DRF's filter backends make it easy to add search to your API.
- For example, to search for books by title:

```
from rest_framework import filters

class BookSearchViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    filter_backends = [filters.SearchFilter]
    search_fields = ['title']
```

- Now `/books/?search=python` will return all books with "python" in the title.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Key Takeaways

- Assigning users to models enables per-user data access and permissions.
- Use custom ViewSet actions for related data endpoints.
- Prefer custom nested serializers over `depth` for clean, secure output.
- Always filter querysets by the authenticated user for privacy.
- Add search with DRF filter backends for a better user experience.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Example

Let’s go do an example together